Lab 08 — Praktijklab: configureren zonder herbouwen, verbergen zonder jezelf iets wijs te maken

Doel: dezelfde image `api:1.0` aansturen met variabelen, de voorrang van de bronnen meten, met eigen ogen zien waar een secret opduikt (logs, `ps`, `inspect`, `env`), en het dan laten verdwijnen met `podman secret` en een secret mount bij het bouwen.

Vereisten — Labs 01 tot 07 afgewerkt. Image `api:1.0` aanwezig (lab 07, stap 0); `postgres:16-alpine` wordt opnieuw opgehaald.

Meegeleverde bestanden (`files/`) — geen: alles wordt aan de terminal aangemaakt.

Stap 1 — Dezelfde image, andere waarden

```
mkdir -p ~/labo-docker/08 && cd ~/labo-docker/08
podman run -d --name api -p 18080:8080 api:1.0 ; sleep 2
curl -s localhost:18080/ ; echo
podman rm -f -t 0 api
podman run -d --name api -p 18080:8080 -p 19090:9090 -e APP_MESSAGE="config via -e" -e
APP_PROFILE=prod -e SERVER_PORT=9090 api:1.0 ; sleep 2
curl -s localhost:18080/ ; echo
curl -s localhost:19090/ ; echo
podman logs api | head -2
```

Observeer eerst `{"message":"Bonjour depuis l'API","profile":"default"}` — de standaardwaarden die in de applicatie gecodeerd zijn —, daarna **niets** op 18080 en `{"message":"config via -e","profile":"prod"}` op 19090, met `API demarree sur le port 9090 (profil prod)` in de logs.

Uitleg. Drie variabelen hebben het bericht, het profiel en de luisterpoort veranderd, zonder rebuild. En `-p 18080:8080` geeft door naar een poort waar niemand meer luistert: `EXPOSE 8080` in de image heeft niets gecontroleerd.

Stap 2 — Alles is zichtbaar

```
podman inspect api --format '{{range .Config.Env}}{{println .}}{{end}}' | grep APP_
podman exec api env | grep APP_
podman exec api sh -c 'tr "\0" "\n" < /proc/1/environ | grep APP_'
podman rm -f -t 0 api
```

Observeer drie keer dezelfde `APP_MESSAGE=config via -e` en `APP_PROFILE=prod`: in de metadata van de container, in de omgeving die `exec` ziet, en in die van proces 1.

```
podman run -d --name api api:1.0 --db-password=Secr3t ; sleep 2
podman logs api | head -1
podman inspect api --format '{{json .Config.Cmd}}'
podman top api args
ps -eo args | grep -c 'db-password=Secr3t'
podman rm -f -t 0 api
```

Observeer `Arguments recus : --db-password=Secr3t` in de **logs**, `["--db-password=Secr3t"]` in de inspect, `java -jar /app/api.jar --db-password=Secr3t` in `top`, en `2` processen op de host die de waarde tonen (de `java` en de `grep` zelf).

Uitleg. Een variabele is leesbaar voor wie toegang heeft tot de engine; een argument is leesbaar voor **iedereen** op de machine, en belandt in de logs. Geen van beide is een kluis.

Stap 3 — Voorrang van de bronnen

```
printf '# configuratie van de API (nooit in git)\nAPP_MESSAGE=config via env-
file\nAPP_PROFILE=staging\n' > api.env
printf 'FROM localhost/api:1.0\nENV APP_MESSAGE="config uit de image" APP_PROFILE=image\n' >
Dockerfile.env
podman build -q -f Dockerfile.env -t api:env .
podman image inspect api:env --format '{{range .Config.Env}}{{println .}}{{end}}' | grep APP_
```

Observeer `APP_MESSAGE=config uit de image` en `APP_PROFILE=image`: standaardwaarden die in de image vastgelegd zijn.

```
podman run -d -p 18080:8080 --name a1 api:env ; sleep 2 ; curl -s localhost:18080/ ; echo ;
podman rm -f -t 0 a1
podman run -d -p 18080:8080 --name a2 --env-file api.env api:env ; sleep 2 ; curl -s
localhost:18080/ ; echo ; podman rm -f -t 0 a2
podman run -d -p 18080:8080 --name a3 -e APP_PROFILE=prod --env-file api.env api:env ; sleep 2
; curl -s localhost:18080/ ; echo ; podman rm -f -t 0 a3
APP_MESSAGE="uit de shell van de host" podman run -d -p 18080:8080 --name a4 -e APP_MESSAGE
api:env ; sleep 2 ; curl -s localhost:18080/ ; echo ; podman rm -f -t 0 a4
```

Observeer achtereenvolgens `"config uit de image"/"image"` (alleen de image), `"config via env-file"/"staging"` (het bestand overschrijft de image), `"config via env-file"/"prod"` (`-e` overschrijft het bestand, hoewel het **vóór** `--env-file` staat), en `"uit de shell van de host"/"image"` (`-e APP_MESSAGE` zonder waarde heeft de variabele van de shell gekopieerd).

Uitleg. De voorrang gaat per categorie, niet per positie: `-e` > `--env-file` > `ENV`. Het bestand `api.env` is de natuurlijke kandidaat voor "een set waarden per omgeving", op voorwaarde dat het buiten Git blijft.

Stap 4 — Ook de JVM leest de omgeving

```
podman run -d --name a5 -e JAVA_TOOL_OPTIONS="-Xmx64m" api:1.0 ; sleep 2
podman logs a5 2>&1 | head -1
podman rm -f -t 0 a5
```

Observeer `Picked up JAVA_TOOL_OPTIONS: -Xmx64m`: de JVM heeft de optie overgenomen zonder dat de `ENTRYPOINT` veranderde.

Uitleg. Dit is de standaardhefboom om het geheugen in te stellen of een agent te koppelen aan een image die je niet herbouwt.

Stap 5 — Een secret van de engine voor PostgreSQL

```
printf 'S3cr3t-pg' | podman secret create db_password -
podman secret ls
podman secret inspect db_password --format '{{.Spec.Driver.Name}}
{{.Spec.Driver.Options.path}}'
podman secret inspect --showsecret db_password --format '{{.SecretData}}'
```

Observeer een ID, `db_password` met de driver `file`, het pad `…/storage/secrets/filedriver`, en — met `--showsecret` — `S3cr3t-pg`: opgeborgen, niet versleuteld.

```
podman run -d --name db --secret db_password -e POSTGRES_PASSWORD_FILE=/run/secrets/db_password
postgres:16-alpine ; sleep 4
podman exec db ls -l /run/secrets/
podman inspect db --format '{{range .Config.Env}}{{println .}}{{end}}' | grep POSTGRES
podman inspect db --format '{{json .Config.Env}}' | grep -c 'S3cr3t'
podman run --rm --network container:db -e PGPASSWORD=S3cr3t-pg postgres:16-alpine psql -h
localhost -U postgres -c 'select current_user'
```

Observeer `-r--r--r-- 1 root root 9 … db_password`, één enkele `POSTGRES_PASSWORD_FILE=/run/secrets/db_password` in de omgeving, `0` keer de waarde in de inspect, en `current_user = postgres`: het wachtwoord is wel degelijk dat van het secret.

```
podman rm -f -t 0 db
podman run -d --name db2 --secret db_password,type=env,target=POSTGRES_PASSWORD postgres:16-
alpine ; sleep 3
podman inspect db2 --format '{{json .Config.Env}}' | grep -c POSTGRES_PASSWORD
podman exec db2 env | grep POSTGRES_PASSWORD
podman rm -f -t 0 db2
```

Observeer `0` in de inspect maar `POSTGRES_PASSWORD=S3cr3t-pg` in de omgeving van het proces: de variant `type=env` dient voor applicaties die geen bestand lezen, ten koste van een waarde die `exec` kan zien.

Uitleg. Het secret gaat noch via de commandoregel, noch via de metadata. Met `_FILE` ziet alleen het proces dat `/run/secrets/db_password` opent het.

Stap 6 — Een secret bij het bouwen

```
printf 'ghp_prive-token' > token.txt
printf 'FROM docker.io/library/alpine\nRUN --mount=type=secret,id=token,required=true wc -c
/run/secrets/token && echo "token gebruikt, niet bewaard"\nRUN ls /run/secrets 2>&1 || true\n'
> Dockerfile.secret
podman build --secret id=token,src=token.txt -f Dockerfile.secret -t secret-demo . 2>&1 | grep
-v '^-->'
podman history --no-trunc secret-demo --format '{{.CreatedBy}}' | head -2
podman run --rm secret-demo ls /run/secrets
```

Observeer `15 /run/secrets/token` tijdens de instructie die het secret mount, daarna `ls: /run/secrets: No such file or directory` vanaf de volgende instructie; in de geschiedenis de regel `--mount=type=secret,id=token,…` **zonder de waarde**; en in de eindimage geen `/run/secrets`.

```
podman build -f Dockerfile.secret -t secret-demo2 . 2>&1 | tail -1
podman build --no-cache -f Dockerfile.secret -t secret-demo2 . 2>&1 | tail -1
```

Observeer dat de build **zonder** `--secret` slaagt (alles komt uit de cache), en daarna, met `--no-cache`, `Error: … secret required but no secret with id "token" found`.

Uitleg. Het secret bestaat alleen tijdens één instructie en komt in geen enkele laag terecht. De cache vraagt niets opnieuw: een verlopen token ontdek je bij de eerste build zonder cache.

Opruimen

```
podman secret rm db_password
podman rmi api:env secret-demo secret-demo2
podman rmi docker.io/library/postgres:16-alpine
rm -f token.txt Dockerfile.secret
podman secret ls ; podman ps -a --format '{{.Names}}'
```

Observeer een lege lijst van secrets en geen enkele container. Hou `api.env` en `Dockerfile.env`: lab 09 hergebruikt `api.env` met Compose.

Wat je nu moet kunnen beweren

- Drie `-e`-opties veranderden bericht, profiel en poort zonder rebuild — en `-p 18080:8080` gaf door naar het niets.
- Een variabele is zichtbaar in `inspect`, `exec env` en `/proc/1/environ`; een argument is **bovendien** zichtbaar in de logs, `top` en de `ps` van de host.
- `-e` > `--env-file` > `ENV`, ongeacht de volgorde; `-e SLEUTEL` alleen kopieert de variabele van de host.
- `Picked up JAVA_TOOL_OPTIONS`: de JVM configureert zich via een variabele.
- `podman secret` + `_FILE`: `0` keer het wachtwoord in `inspect`, een bestand `444` in `/run/secrets/`; `--showsecret` bewijst dat het niet versleuteld is.

- `--mount=type=secret`: `15 /run/secrets/token` tijdens de instructie, `No such file or directory` daarna, niets in `history`.
- Zonder `--secret` slaagt de gecachete build; `--no-cache` → `secret required but no secret with id "token" found`.